一、C 语言部分（赵友先老师）

1.1 请简述 static 关键字用途

1）static 修饰局部量

局部变量的生命周期得到延长。局部变量默认是程序运行的时候，在栈中分配空间的，这样在函数结束的时候，局部变量对应的内存已经被释放掉了。当用 static 修饰局部变量的时候，static 修饰的局部变量在分配的空间在静态区区域(data/bss 段)，这样的局部变量，不会在函数调用结束的时把对应的空间释放掉。

局部变量值的得以保留，不论函数调用多少次局部变量只初始化一次。局部变量如果没有赋初值的时候，默认值为随机值(程序运行的时候，在栈中开辟空间，此时分配内存中存放的值无法确定)。static 修饰的局部变量默认值为 0，并且这个变量的值具有继承性(每次调用的时候，static 修饰的变量不会每次重新赋值，而是接着使用上一次的值)。

2）static 修饰函数/全局变量

static 修饰的函数/全局变量的作用范围被限定在本文件中。默认情况下函数名或全局变量是全局可见的，在头文件中经常使用关键字 extern 声明。但当用 static 修饰它们的时候，暗示该函数/变量只是在本文件中有效，就不能再在头文件中用 extern 关键声明。

1.2 请简述 const 关键字用途

const 修饰的数据类型是指常类型，常类型的变量或对象的值是不能被更新的。const 关键字的作用主要有以下几点：

1）可以定义 const 常量，使其具有不可变性。例如：

```
const int len = 100;
int Array[len]; //允许用常量初始化数组
```

```
int len = 100;
int Array[len]; //不允许用变量初始化数组
```

2）便于进行参数类型检查，消除了一些隐患。例如：

```
void func(const int i)
{
    ... //形参 i 是一个常量，不允许被修改
}
```

```
void func(const int i)
{
    i = 10; //错误，i 不允许被修改
}
```

华清远见武汉中心嵌入式学科面试宝典

3）const 修饰普通变量

```
const int a;  //ok!
int const a;  //ok!
int a const;//error! 语法错误
a = 5;//error! 不能改 a 的值
```

4）const 修饰指针变量

```
int num;

const int *a;  //常量指针写法，指向常量的指针
int const *a;
const int const *a;
a = &num; //ok! 修改指针指向
*a = 20; //error! 不能间接寻址

int * const b = &num;  //指针常量写法，指针同时是一个常量
b = null; //error! 不能修改指针指向
*b = 20; //ok! 间接寻址

const int * const c = &num;  //常用写法
int const * const c = &num;
const int const * const c = &num;
c = null; //error! 不能修改指针指向
*c = 20; //error! 不能间接寻址
```

5）可以节省空间，避免不必要的内存分配。例如：

```
#define PI 3.14159 //常量宏

const doulbe pi = 3.14159; //此时并未将 pi 放入 ROM 中 i 放入 ROM 中
double i = pi; //此时为 pi 分配内存，以后不再分配!
double j = pi; //pi 没有内存分配
double I = PI; //编译期间进行宏替换，分配内存
double J = PI; //编译期间进行宏替换，又一次分配内存!
```

编译器通常不为普通 const 常量分配存储空间，而是将它们保存在符号表中。const 定义常量从汇编的角度来看，只是给出了对应的内存地址，而不是象#define 一样给出的是立即数，所以 const 定义的常量在程序运行过程中只有一份拷贝，而#define 定义的常量在内存中有若干个拷贝。这使得它成为一个编译期间的常量，没有了存储与读内存的操作，使得它的效率也很高。

6）为函数重载提供了参考（C++部分）

华清远见武汉中心嵌入式学科面试宝典

```
class A
{
    void func(int i) { //一个函数
        ...
    }
    void func(int i) const //函数重载
    {
        ...
    }
};
```

1.3 请简述 volatile 关键字用途

Volatile 中文含义表示"易变的"，它是 C 语言中用来修饰变量的一个关键字。

需要知道的是：每个编译器都有自己的优化级别。编译器进行编译成机器码时，如果它发现这个变量已经在前面的代码执行过程中（已将变量加载到寄存器），那么后面在使用这个变量的时候，就不会再从内存中重新读值，而是直接使用寄存器中的值。这样做目的是提高程序运行的效率，但同时也会带来一个很隐晦的 bug。我们来看看如下代码：

```
short flag;

__interrupt  handler()
{
    flag = 1;
}
```

```
void test()
{
    do1();
    while(flag == 0);
    do2();
}
```

这段程序等待内存变量 flag 的值变为 1 之后才运行 do2()。变量 flag 的值由别的程序更改，这个程序可能是某个硬件中断服务程序。例如：如果某个按钮按下的话，在按键中断程序中修改 flag 为 1，这样上面的程序就能够得以继续运行。但是，编译器并不知道 flag 的值会被别的程序修改，因此在它进行优化的时候，可能会把 flag 的值先读入某个寄存器，然后等待那个寄存器变为 1。如果不幸进行了这样的优化，那么 while 循环就变成了死循环，因为寄存器的内容不可能被中断服务程序修改。为了让程序每次都读取真正 flag 变量的值，就需要定义为如下形式：

```
volatile short flag;
```

需要注意的是，没有 volatile 也可能能正常运行，但是可能修改了编译器的优化级别之后就又不能正常运行了。因此经常会出现 debug 版本正常，但是 release 版本却不正常的问题。所以为了安全起见，只要是等待别的程序修改某个变量的话，就加上 volatile 关键字。

最后一句话总结它：volatile 关键字的主要用途是为了防止编译器优化，告诉编译器，在使用它修饰

华清远见武汉中心嵌入式学科面试宝典

的变量时，必须每次从内存中重新读值，而不是直接使用上一次从内存中加载到寄存器中的值。

1.4 库函数 strcpy()和 memcpy()的区别

	char *strcpy(char *dest, const char *src);	void *memcpy(void *dest, const void *src, size_t n);
操作对象	只能是字符串数组	任意类型的数据
功能特性	将指针 src 指向的数据依次拷贝到 dest 指向的单元，直至'\0'结束；	将指针 src 指向的数据依次拷贝到 dest 指向的单元，拷贝 n 个字节数据；
注意事项	1）src 指向的字符串不能长于 dest 指向的空间，不然会覆盖其他数据； 2）src 指向的数据块必须有'\0'结尾，不然会数组越界；	1）src 指向数据不得长于 dest 指向的空间，不然会覆盖其他数据； 2）n 不得大于 src 直线的数据块长度，不然会数组越界；

1.5 指针常量与常量指针的定义及区别

	指针常量	常量指针
定义	顾名思义，是一个常量；即指针所指向的位置不能改变，即指针本身是一个常量。 定义时：先写指针：type * 再写常量：const name；连起来就是 type * const name。	指向常量的指针，简称常量指针。常量指针本质是指针，且这个指针是一个指向常量的指针，指针指向的变量的值不可通过该指针修改，但是指针指向的值可以改变。定义： 先写常量 ：const ，再写指针：type * name；连起来就是 const type *name。
用法举例	int * const p = &a; 当你希望一个指针在程序的生命周期内始终指向同一个地址，但允许修改该地址上的数据时，可以使用指针常量。例如，当你需要一个固定指向某个全局变量的指针，并且不希望这个指针被重新赋值为其他地址时。	const int * p; int const * p; 当你需要一个指针来读取某个值，但不希望修改这个值时，可以使用常量指针。这在函数参数传递时非常有用，可以防止函数内部修改传入的参数值。例如，当你需要传递一个数组给函数，但不希望函数修改数组中的元素时。
注意	1）指针常量必须在声明的同时对其初始化，不	1）可以将一个变量的地址赋值给常量指针，防止

华清远见武汉中心嵌入式学科面试宝典

事项	允许先声明一个指针常量随后再对其赋值； 2）它是个常量！指针所指向不可以变化，但是指向的地址所对应的内容可以变化； 3）在题目中判断时，可以去掉数据类型再判断，const 作用在指针还是变量上面。	其修改内存数据； 2）指针还可以指向别处，因为指针本身只是个变量，可以指向任意常量的地址；

1.6 多文件编程有什么好处

多文件编程是必要的，甚至是必须的。简单来说就是把功能相近或者相关的函数变量定义添加到.c 和.h 文件中，这样的两个文件称为一个模块。其好处至少可以说出以下几点：

1）方便代码复用：模块化的代码可以很方便的迁移到其他项目中去，改写模块比重写模块更快；

2）方便分工合作：各个功能模块分成多个文件同时编辑，可以有效的提高团队开发的分工协作效率；

3）方便后续维护：项目源码交接时，不管是客户还是同事，对于划分明确的文件是不会排斥的；

4）保证了库支持：个人函数库都是由 c 文件产生库，采用“库文件+h 文件”的形式，可以实现快速编译和保护源码的效果。

1.7 有了解过 MISRA 吗，汽车行业 C 编码规范

MISRA (The Motor Industry Software Reliability Association)，中文名称为汽车工业软件可靠性联会，是英国的一个跨国汽车工业协会，其成员包括了大部分欧美汽车生产商。其核心使命是为汽车工业提供服务和协助，帮助厂方开发安全的、高可靠性的嵌入式软件。MISRA C 则是由 MISRA 提出的针对嵌入式 C 语言开发标准，目的是提高嵌入式系统的安全性、可读性和可移植性。

MISRA C 一开始主要是针对汽车产业，后面他产业也逐渐开始使用 MISRA C，包括航天、电信、国防、医疗设备、铁路等领域中都已有厂商使用 MISRA C。通常认为，如果能够完全遵守这些标准，则该嵌入式 C 代码就是易读、可靠、可移植和易于维护的。因此，很多嵌入式开发者都以 MISRA C 来衡量自己的编码风格，比如著名的 uC/OS 就称自己 99%的代码遵循 MISRA C 编码规范标准。因此，作为嵌入式开发者，不论哪个行业，都应以产品安全性为前提，保证可靠性、可读性、可移植性，也就是保证自身代码尽可能符合 MISRA C。

以下为 MISRA C 2004 规范标准：

规则 1.1（强制）：所有的代码应该遵守 ISO 9899:1990“Programming Language C”

规则 1.2（强制）：只有当具备统一接口的目标代码的时候才可以采用多种编译器和语言。

华清远见武汉中心嵌入式学科面试宝典

规则 1.4（强制）：检查编译器 / 连接器以确保支持 31 一个有效字符，支持大小写敏感。

规则 2.1（强制）：汇编语言应该封装起来并且隔离。

例如：#define NOP asm(“NOP”)

规则 2.2（强制）：源代码只能采用 /* …*/风格的注释。

规则 2.3（强制）：字符序列 /* 不能在注释中使用。

注:C 语言不支持注释的嵌套即使一些编译器支持这个语言扩展。

规则 2.4（建议）：代码段不能注释掉。

注：应采用 #if 或者#ifdef 来构成一个注释。

规则 3.3（建议）：编译器对于整数除法运算的实施应该写入文档。

例： -5/3 = -1 余-2 有些编译器结果是 -2 于＋1。

规则 4.1（强制）：只能使用 ISO 标准定义的字符集。

规则 5.1 （强制）：在内部范围的标识符不能和外部的标识符用同样的名字，因为会隐藏那个标识符。

```
int16_t i:
void f()
{
    int16_t i;
    i = 3;
}
```

规则 5.2（强制）：typedef 名称只能唯一，不能重复定义。

规则 5.4（强制）：标记名应该是唯一的标识符。

规则 5.7（建议）：标识符不能重复使用。

规则 6.1（强制）：char 类型只能用来存储使用字符。

规则 6.2（强制）：signed 和 unsigned char 只能用来存储和使用数据值

规则 6.3（建议）：对于基本数据类型，必须使用 typedef 显式标识出数据长度。例：

typedef signed char int8_t;typedef unsigned int uint16_t;

注意：还有很多规则，这里就不一一介绍了。

1.8 对比如下两种结构，说说他们的好处还有坏处

```
if(条件) {
    for(i=0; i<n; i++) {...}
}
else {
    for(i=0; i<n; i++) {...}
```

```
for(i=0; i<n; i++) {
    if(条件) {
        ...
    }
    else {
```

华清远见武汉中心嵌入式学科面试宝典

} 	... } }

结构 1：根据判断条件执行不同的循环，效率高于结构 2，但代码空间占用较大；

结构 2：每一轮循环都要执行判断条件，效率低于结构 1，但代码空间占用较小，可读性也较好；

1.9 源文件到可执行文件的流程

预处理	进行头文件的展开，宏的替换，去除注释等预先处理步骤，不检查语法错误，生成原始文件 demo.i；
	$ gcc -E demo.c //预处理文件 demo.c
	$ gcc -E demo.c -o demo.i

编译	检查代码的语法结构、语法错误等，将原始文件生成汇编文件.s $ gcc -S demo.i -o demo.s
汇编	将汇编文件转换为计算机能识别的机器码（二进制文件），生成.o 文件。 gcc -c demo.c -o demo.o //将源码文件直接进行汇编处理，生成目标文件 gcc -c demo.s -o demo.o //将汇编文件进行汇编处理，生成目标文件
链接	将目标文件、依赖库文件等链接组成一个可执行文件 gcc demo.o -o demo //将目标文件链接生成可执行文件,可执行文件名自定义。

1.10 #define 与 const 的区别

	#define	const
作用	#define 是预处理指令，它用于定义宏。	const 关键字用于定义具有特定类型的常量变量

华清远见武汉中心嵌入式学科面试宝典

<table>
<tr><td>区别</td><td>1）#define 是预处理指令，它在编译之前就被处理了，不占据内存空间。
2）它用于定义宏，这些宏在编译时会被替换为它们对应的值或表达式。
3）#define 定义的常量没有类型，它只是简单的文本替换。
4)const 常量更适合用于定义需要类型检查的常量，而 #define 更适合用于条件编译和定义编译时常量。</td><td>1）这些变量在编译时分配内存，并且它们的值在运行时不能被改变。
2）const 定义的常量可以用于数组、指针等更复杂的数据结构中，并且它们具有类型安全。
3）const 定义的常量具有类型安全，而 #define 定义的宏没有类型。
4）const 常量具有作用域的概念，它们遵循块作用域（局部）或文件作用域（全局）的规则。
而 #define 定义的宏没有作用域的概念，除非使用 #undef 取消定义。</td></tr>
<tr><td>例子</td><td colspan="2">#include <stdio.h>

#define MAXSIZE 100 // 预处理宏定义
const int max_size = 100; // 定义具有类型的常量

int main()
{
int array[MAXSIZE]; // 使用#define 定义的宏
int b[max_size]; // 在 c 语言中错误用法，将会报错。C++中可以。
}
在这个示例中，MAXSIZE 是使用 #define 定义的宏，而 max_size 是使用 const 定义的常量。
在 c 语言中前者可以用来定义数组的大小，但 const 定义的常量不可以；c++中以上两种都可以，</td></tr>
</table>

华清远见武汉中心嵌入式学科面试宝典

	const 提供了更好的类型安全性和可维护性。

1.11 #define 与 typedef 的区别

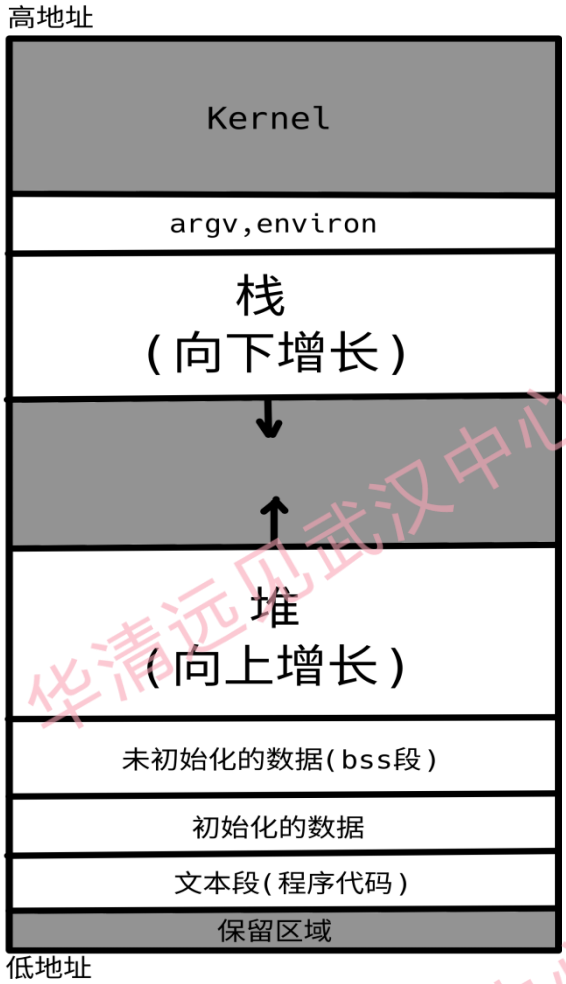
	#define	typedef
作用	#define 是预处理指令，它用于定义宏。	typedef 是一个关键字，用于为数据类型创建一个新的别名。
区别	从作用就有着显著的区别； 1）它没有类型，主要用于常量定义、条件编译等场景。	1）typedef 是关键字，对已经存在的数据类型取别名。 在编译阶段处理，会进行类型检查，只能在定义的作用域内使用。

例子	`#include <stdio.h>`
	`#define MAX_SIZE 100 // 预处理宏定义一个常量`
	`typedef int my_int; // 为 int 创建一个新的别名 my_int`
	`int main()`
	`{`
	`int array[MAX_SIZE]; // 使用宏定义的常量作为数组大小`
	`my_int value = 10; // 使用 typedef 创建的别名`
	`printf("Array size: %d\n", MAX_SIZE);`
	`printf("Value: %d\n", value); return 0;`
	`}`	

1.12 C 语言内存模型以及各自的特点

模型如图：

华清远见武汉中心嵌入式学科面试宝典



各自的特点：

Kernel:	内核是 C 内存模型中的关键组成部分，它负责管理系统的资源、提供对硬件的访问接口，并提供系统调用接口供用户程序使用。内核在操作系统中扮演着非常重要的角色，确保计算机系统的正常运行和资源的有效管理。
栈的特点:	系统自动分配和自动释放，栈在程序中的主要作用是管理函数的调用和返回。当一个函数被调用时，它的局部变量和其他相关信息被压入栈中，函数执行完毕后，这些信息被弹出栈。这样可以确保函数的局部变量和上下文信息在函数调用过程中的正确性和独立性。 此外，栈还可以用于存储临时数据、递归算法、表达式求值等。栈的大小相对较小，但访问速度较快，因此在需要快速分配和释放内存的场景下，栈是一个常用的数据结构。
堆:	**程序员手动分配**：堆内存的大小可以在程序运行时动态地增长或缩小，根据程序的需要进行内存分配和释放。 **随机访问**：堆中的内存块可以通过指针进行随机访问，程序可以根据需要在堆中分配和访问任意大小的内存块。

华清远见武汉中心嵌入式学科面试宝典

	持久性：堆中分配的内存块在分配后会一直存在，直到显式释放或程序结束。 **不连续分配：**堆中的内存块不一定是连续的，可以是散布在堆内存区域的不同位置。
BSS:	BSS 段储存的是未初始化的全局变量或初始化为 0 的全局变量， BSS 段不占据执行文件空间，但占据程序运行时的内存空间。 执行期间必须将 BSS 段内容全部设为 0
data 段:	rodata 段 rodata 段存储常量数据，比如程序中定义为 const 的全局变量，#define 定义的常量，以及诸如“Hello World”的字符串常量。只读数据，存储在 ROM 中。 const 修饰的全局变量在常量区；const 修饰的局部变量只是为了防止修改，没有放入常量区。 编译器会去掉重复的字符串常量，程序的每个字符串常量只有一份。 wqdata 段 data 存储已经初始化的全局变量，属于静态内存分配。（注意：初始化为 0 的全局变量还是被保存在 BSS 段） static 声明的变量也存储在数据段，链接时初值加入执行文件。
代码段:	text 段存放程序代码，运行前就已经确定（编译时确定），通常为**只读。**

1.13 编写 atoi 函数（字符串转整型函数）的实现 函数原型：int My_Atoi(const char *str);

```
#include<stdio.h>
#include<stdlib.h>
#include<ctype.h>
/*atoi()函数的功能：将字符串转换成整型数；atoi()会扫描参数 nptr 字符串，
跳过前面的空格字符，直到遇上数字或正负号才开始做转换，
而再遇到非数字或字符串时（'\0'）才结束转化，并将结果返回（返回转换后的整型数）。
{
    int flag = 1;
    //判断是否是 NULL 指针或空白字符串
```

华清远见武汉中心嵌入式学科面试宝典

```
    if (str == NULL && *str == '\0')
        return -1;
    //过滤空白字符
    while (isspace(*str))
    {
        str++;
    }
    //使用标志 flag 来判断数字正负
    if (*str == '+')
    {
        flag = 1;
        str++;
    }
    else if (*str == '-')
    {
        flag = -1;
        str++;
    }
{
    int flag = 1;
    //判断是否是 NULL 指针或空白字符串
    if (str == NULL && *str == '\0')
        return -1;
    //过滤空白字符
    while (isspace(*str))
    {
        str++;
    }
    //使用标志 flag 来判断数字正负
    if (*str == '+')
```

华清远见武汉中心嵌入式学科面试宝典
华清远见武汉中心嵌入式学科面试宝典
华清远见武汉中心嵌入式学科面试宝典

```
    {
        flag = 1;
        str++;
    }
    else if (*str == '-')
    {
        flag = -1;
        str++;
    }
```

华清远见武汉中心嵌入式学科面试宝典
华清远见武汉中心嵌入式学科面试宝典
华清远见武汉中心嵌入式学科面试宝典